Calendar Application – Chronos

Delegation of Tasks for Deliverable #2

Presentation Slides	–	Everyone
Presentation Recording	–	Everyone
Scheduling and Estimation Studies		
Project Scheduling	–	Kayla Vu
Cost, Effort and Pricing Estimation	–	Jonfernando Alfaro, Kayla Vu
Estimated Cost of Hardware Products	–	Andrew Cross
Estimated Cost of Software Products	–	Radhika Sharma
Estimated Cost of Personnel	–	Deeksha Patnam
Test Plan		
Describing test plan	–	Abhay Srivatsa
Write code	–	Sourish Pasula
Define test cases and results	–	Abhay Srivatsa
Comparison of Work	–	Kayla Vu, Deeksha Patnam
Conclusion	–	Kayla Vu

Assumptions

The main team of software developers that are actually creating and developing the calendar application is still a team of *7 members* as the software process model that we're deploying for this project is mainly the Incremental Process Model which is most efficient in smaller groups with skilled developers.

The team is contacted by a mid-sized company to create a mobile calendar application to integrate into their system. Several resources [1, 2] agree that a mid-sized company usually consists of around 1,500 to 2,000 employees. For companies with also more than 1,000 employees but less than 5,000, these resources approximate that there should be around 38 IT employees within this range of employees. The annual company revenue is also estimated to be around $38.5 million to $1 billion.

Scheduling and Estimation Studies

a) Project Scheduling

i) Please provide a start date, end date by giving justifications about your estimation.

Under the assumptions above, the main software development team still consists of 7 members of software developers deploying an agile method.

- *Incremental Process Model* with specifically a parallel development model for delivering features of the calendar application.
- *Iterative Development* in which each increment will be refined and improved upon with each cycle based on tests and user feedback.

We've outlined a project schedule below, starting immediately in 2026 on January 5th. Companies like Ditstek Innovations Pvt. Ltd. (DITS) [3] outline different stages that projects usually entail and their estimated times such as the requirements definition, architecture design, development phase etc. that we have decided to include in our own schedule.

Start Date: January 5

1. **Requirements Definition** (~1-3 weeks: **2 weeks**)

   During this stage, our team of developers would be investing in time planning a roadmap by first getting ideas and requirements from the company, getting user/employee feedback through interviews and surveys, and document the requirements.

2. **Architecture Design** (~3-8 weeks: **4 weeks**)

   After we receive the feedback and requirements, we would be planning the actual roadmap. A clear plan would be produced during this stage to ensure that developers and stakeholders both understand what the product is and what will happen in future increments. This phase will consist of developing mock designs, defining milestones and developer roles.

3. **Development Phase** (~8-20 weeks: **10 weeks**)
   a. Incremental Process – add features at a time (add/delete events, change calendar view etc.) until all features are implemented and requirements are fulfilled

4. **Verification & Validation Tests** (~3-6 weeks: **4 weeks**)
   a. Iterative Development – test out features implemented to ensure no security issues, bugs, or performance issues and to gain user feedback for improvement
   b. This part and the development phase will be going back and forth

5. **Deployment** (~1-2 weeks: **2 weeks**)

   During this phase, we'll prepare for the official launch (finalize configurations, notify employees etc.) and train company IT workers and other personnel for future support and maintenance.

End Date: June 12 (total 22 weeks = ~5-6 months)

The timeline of this project estimates a total of 22 weeks or 5-6 months as DITS [3] approximates that simpler applications usually take 2-4 months and medium complexity applications such as content management systems take around 6-12 months. Although a calendar application is fairly basic, we decided to account for any errors that may delay project completion such as security issues or added requirements to implement. This is because many sources [4, 5] argue that it's better to overestimate the time needed for a project rather than underestimate and possibly fail to deliver a completed application on time.

ii) **Also provide the details for:**

1) **Whether weekends will be counted in your schedule or not**
   (a) Weekends will not be counted in the schedule
2) **What is the number of working hours per day for the project?**
   (a) 8 hours a day x 5 days a week = 40 hour work week total
   (b) National Holidays will not be considered in the count either

b) Cost, Effort and Pricing Estimation

i) Method used to calculate the estimated cost and in turn price of the project

The method we decided to deploy was the function point (FP) which measures the functional size of our calendar application based on the application's user-visible features instead of its lines of code. For our project, this is much preferred as we're planning the estimation before our lines of code are even written, especially under the assumption that we need to plan and work on a roadmap with our client company first. The FP method helps us work and prepare before coding begins, allows us to focus on our core features first, estimate size early in the project lifecycle and convert this functional size into effort, cost and duration once we calculate the FP total.

ii) Function Point (FP) Application

Assuming that our application will run similar to a google calendar app, the app will contain these core features:

- Event creation, editing and deletion
- Recurring or unique events
- Notifications
- Authentication for accounts
- Multi-device sync
- Time-zone and holiday handling
- Features that allow for collaboration and sharing
- Mobile

Our Calculations

FP: COMPUTE GROSS FUNCTION POINT

	Function Category	Count	Complexity			Count x Complexity
			Simple	Average	Complex	
1	**External Inputs** Create Event Edit Event Delete Event Event Reminders User Authentication User Login / logout Collaborators Decline/accept invitations	8	3	4	6	32
2	**External Outputs** Push notifications Email reminders Daily/Weekly agenda Event summary	4	4	5	7	20
3	**External Queries** View Calendar month/week/day View event details Search Events Load shared calendars	4	3	4	6	16
4	**Internal Logical Files** User Table Events Table Calendars Table Notifications	4	7	10	15	40
5	**External Interface Files** Cloud email provider Cloud storage for	4	5	7	10	20

	dataset Cloud storage for backend and frontend Authentication provider					
					Gross Function Point (GFP):	128

Notes for choosing complexity:

- Dependent on the number of data element types (DET) which are the unique data fields the user can input, view or modify
  - EX: When you create an event the DETs would be title, data, time, location, description, reminder settings which are 5-7 DETs
  - More DETs means higher complexity
- And also dependent on the number of file types references number of file types references (FTR) which are logical data structures that the function must access
  - EX: When you create an event, the system can access the events table, user table and calendars table which are 2-3 FTRs
  - More FTRs means higher complexity

FP: PROCESSING COMPLEXITY (PC)

	General System Characteristic	PC Rating (0-5)	Reason
1	Does the system require reliable backup and recovery?	3	Average necessity; backups are needed for user data but it’s not life-critical
2	Are data communications required?	4	A little more important than average (cloud sync, push, email notifs etc.)
3	Are there distributed processing functions?	3	Client & server, mobile clients (average)
4	Is performance critical?	3	Average; needs real-time viewing/notification and responsiveness

5	Will the system run in an existing, heavily utilized operational environment?	2	Does not need an extreme load initially
6	Does the system require online data entry?	4	Above average necessity; users frequently perform calendar actions like add/delete events
7	Does the online data entry require the input transaction to be built over multiple screens or operations?	2	Below average; some multi-step flows but it's not heavy
8	Are the master files updated online?	4	Above average necessity; events/calendars/users are updated online very frequently
9	Are the inputs, outputs, files or inquiries complex?	3	some complexity with sharing but overall average
10	Is the internal processing complex?	3	Moderate processing
11	Is the code designed to be reusable?	2	Not a major focus
12	Are conversion and installation included in the design?	1	Minimal conversion/ installation effort
13	Is the system designed for multiple installations in different organizations?	1	Made specifically for one mid-sized company to be integrated into their system
14	Is the application designed to facilitate change and ease of use by the user?	4	UX and maintainability prioritized with frequent changes and updates expected
		Total Degree of Influence (TDI)	39

FP: COMPUTE PROCESSING COMPLEXITY ADJUSTMENT (PCA)

PCA = 0.65 + 0.01(PC1 + PC2 + … + PC14)
PCA = 0.65 + 0.01(39) = 1.04

FP: FUNCTION POINT CALCULATION

FP = GFP x PCA
FP = 128 x 1.04 = 133.12

FINAL CALCULATIONS FOR COST, EFFORT AND PRICING

<table>
<tr><th></th><th>CALCULATION</th><th>FINAL TOTAL</th></tr>
<tr><td>FP Calculation</td><td>128 x 1.04 =</td><td>133.12</td></tr>
<tr><td colspan="3">Idealistic Development</td></tr>
<tr><td>Effort</td><td>* Assumption: 10 hours per FP (typical mid-range assumption)

133.12 * 10 hours =</td><td>~1331.2 person hours = 1.82 person months</td></tr>
<tr><td rowspan="2">Project Duration</td><td>* Team size = 7 software developers
7 x 40 hr/week = 280 hr/week

1331.2/280 =</td><td>~ 4.8 weeks or 1.1 months</td></tr>
<tr><td colspan="2">Issue:
● This is an idealistic plan that only accounts for only the coding-related tasks and excludes the other phases of the project development and doesn’t account for any issues that may arise during development</td></tr>
<tr><td>Costs</td><td>* Using Dallas pay rates for mobile software developers [6], the average salary is approximately $11,000 per month

1.82 person months x $11,000 =</td><td>~ $20,020</td></tr>
<tr><td colspan="3">Alternative (More realistic) Project Development Schedule</td></tr>
<tr><td>Effort</td><td colspan="2">* Typical multiplier range: 2.5x to 4x (to include requirements, design, QA, deployment, etc.)

- Lower realistic estimate (2.5x): 1331.2 * 2.5 = 3328.0 hours = 4.55 months
- Mid realistic estimate (3.0x): 1331.2 * 3.0 = 3993.6 hours = 5.47 months
- Higher realistic estimate (4.0x): 1331.2 * 4.0 = 5324.8 hours = 7.29 months</td></tr>
</table>

	- Reported realistic range: ~3300 - 5300+ hours
Project Duration	- Using 2.5x effort (3328 h): 3328 / 280 = 11.8857 weeks ≈ 11.9 weeks (~2.8 months) - Using 3.0x effort (3993.6 h): 3993.6 / 280 = 14.2629 weeks ≈ 14.3 weeks (~3.3 months) - Using 4.0x effort (5324.8 h): 5324.8 / 280 = 19.016 weeks ≈ 19.0 weeks (~4.4 months) With the conservative calculations of 4.4 months, it aligns more similarly with our project schedule calculations as our project schedule approximates 5-6 months worth of work (which also accounts for an added 1-2 months if any issues were to occur).
Costs	- 2.5x effort: 4.55 months x $11,000 = $50,050 - 3.0x effort: 5.47 months x $11,000 = $60,170 - 4.0x effort: 7.29 months x $11,000 = $80,190 * This value may vary in further calculations depending on the position of the employee on the software development team and their specialization and excludes other external costs like resources and equipment.

c) Estimated Cost of Hardware Products

For the professional development of a mobile calendar application targeting both iOS and Android platforms for a mid-sized company with annual revenue of $38.5 million to $1 billion, the following hardware configuration provides optimal performance and comprehensive testing coverage.

1. Development Workstations

Professional-grade equipment for the 7-member development team:

- 7x MacBook Pro 16" (M4 Max chip, 36GB RAM, 1TB SSD) - $3,499 each [7]
- Optimal for both iOS development (Xcode required) and Android development with exceptional performance

Workstations Total: $24,493

2. Testing Devices

Physical device testing ensures compatibility, performance, and user experience across different devices, screen sizes, and OS versions [8, 9, 10].

iOS Testing Devices:

- 2x iPhone 16 Pro (latest flagship, iOS 18) - $999 each [11]
- 2x iPhone 15 (previous generation) - $799 each [11]
- 2x iPhone SE (3rd gen) (budget device, smaller screen) - $429 each [11]
- 2x iPad (10th gen) (tablet testing) - $349 each [12]

iOS Subtotal: $4,854

Android Testing Devices:

- 2x Google Pixel 9 (stock Android, latest OS) - $799 each [13]
- 3x Samsung Galaxy S24 (popular flagship) - $799 each [14]
- 2x Samsung Galaxy A54 (mid-range device) - $449 each [14]
- 1x OnePlus 12 (alternative flagship) - $799 [14]
- 1x Motorola Edge 2024 (budget testing) - $599 [14]
- 2x Samsung Galaxy Tab S9 (Android tablet) - $799 each [14]

Android Subtotal: $7,146

Testing Devices Total: $12,000

3. Peripherals & Accessories

Professional peripherals enhance productivity and provide ergonomic workspaces:

- 7x 32" 4K Professional Monitors (Dell UltraSharp U3223QE) - $650 each [15]
- 7x Premium Wireless Keyboard & Mouse Combos (Logitech MX Keys + MX Master 3S) - $200 each [16]
- 7x Adjustable Laptop Stands (Rain Design mStand) - $60 each
- 7x Thunderbolt 4 Docks (CalDigit TS4) - $400 each [17]

Peripherals Total: $6,500

4. Development & Testing Infrastructure

Local infrastructure for enhanced security, faster access, and greater control:

- Dell PowerEdge T550 Development Server (Intel Xeon, 64GB RAM, 4TB RAID storage) - $3,500 [18]
- Synology DS1621+ Network Attached Storage (6-bay, 24TB total storage) - $1,500 [19]

Infrastructure Total: $5,000

5. Additional Hardware

Supporting equipment for device management and data protection:

- 3x Professional Multi-device Charging Stations (20-port) - $200 each
- 7x External SSD Drives (2TB Samsung T7) - $150 each [20]
- Premium Cables & Adapters (Thunderbolt 4, USB-C, Lightning) - $550

Additional Hardware Total: $1,200

Total Hardware Cost Summary

Category	Cost
Development Workstations	$24,493
Testing Devices	$12,000

Peripherals & Accessories	$6,500
Infrastructure	$5,000
Additional Hardware	$1,200
Subtotal	**$49,193**
Contingency (10%)	**$4,919**
TOTAL ESTIMATED COST	**$54,112**

The professional configuration totaling **$54,112** provides optimal development equipment for the 7-member team, comprehensive testing coverage across 19 devices [8, 9, 10], professional-grade peripherals for enhanced productivity [21] [22], and enterprise infrastructure for secure development. The 10% contingency accounts for price fluctuations and unexpected needs [23]. This investment ensures maximum efficiency for the 22-week project and reflects professional standards appropriate for a company with annual revenue of $38.5 million to $1 billion.

d) Estimated Cost of Software Products

When it comes to creating a calendar software, the range of the prices will depend on the features that we include into the software and just overall the development of the software. The pricing when it comes to developing a calendar software varies as the price ranges to "$30k-$150k".[24] If we were deciding to make a simple calendar (which will include the important parts of the calendar) then the price will be between "$30k-$60k".[24] In the calendar software, if we included some features which can make the calendar more usable then it ranges $60k-$100k.[24] Then if we provide more features that are more advanced and more analytics then the prices can go more than $150k. Some factors that can affect the price of the app are cloud hosting, database, frontend and backend framework and including libraries.

1. Calendar Libraries

First, the calendar libraries play an important part when it comes to our software because it can be used to save the developers time when it comes to developing everything of the calendar software. Since we are deciding to create it as an app for IOS and Android, we would need to use different libraries such as "FSCalendar, JTAppleCalendar, CalendarKit" for IOS and "CosmoCalendar, SlyCalendarView, AgendaCalendarView"[26] for Android. If we do include a web version as well then there are other libraries specified towards the web version. Most of the calendar libraries can be accessed free through GitHub.

2. Cloud Hosting

Next is cloud hosting, which is used to make the websites and apps available on the internet by using the cloud. Usually the costing of a cloud hosting is a monthly payment or a "pay-as-you-go model" depending on how much usage there is.[28] The advantages of cloud hosting is that it "allows freedom to use the appropriate solution that any situation requires by provisioning the parameters of the machines across the network."[28] Another advantage of cloud hosting is providing good security when it comes to keeping our data safe and secure. Some cost of cloud hosting from certain companies such as Google is providing Google cloud products with $300 in credit to use to spend over the next 90 days. It also states that there will be no automatics charges but the charges will start when it activates, pre-pay or pay-as-you-go method. The Google Cloud products will include API, services and other products that are given to the user.

3. Database

Another important thing is the database in which it will store all the information in a calendar software. Some softwares that we can use when it comes to storing data such as mySQL and PostgreSQL. Having a database can store all the events and details such as the time, date, location and other stuff about the event. The pricing for this might depend on the fact that the mySQL has a free version and a monthly payment version as well.

4. Front-end & Back-end Development

Lastly, having a front-end and back-end development which will help the software when it comes to the user interaction with the software and managing our software's application functionality. With front-end development this means that it uses "the GUI (graphical user interface) that our user will be using to interact" with our calendar software such as the reaction pressing a new event on the calendar app and it will react which is showing all the new event details that the user can type out. [27] The back-end development is using a programming language to deal with the data and communication when it comes to the software and the database. When it comes to both front and back end development, the pricing can depend on the "project scope, location of the team and the expertise level."[27] The front-end development pricing such as "basic business website is between $2000-$5000, Web Application $15000 - $50000" and others project based pricings.[25] The back-end development has similar range prices as the front-end development.

e) Estimated Cost of Personnel

We have divided our personnel costs into two parts - Development team and post-launch team

Personnel Requirements for Development

We are estimating a total team size of 7 people from all domains.

1. Frontend Developers - 2 people

There is a need to build different calendar views - daily, weekly, monthly, agenda, event creation/editing interfaces, and a responsive design across different screen sizes (320×568 px and larger). *We are estimating an hourly rate of $40-50, and each person would spend 800-1000 hours (approx.). The cost per developer turns out to be $32,000-50,000. Hence for 2 developers, it would be $64,000-100,000.*

2. Backend Developers - 2 people

We are implementing MVC Model and Controller components, event data management, recurring event logic. Moreover tasks like sharing permissions, authentication (OAuth), encryption, API development, and database design would be assigned to the backend team. *We are estimating an hourly rate of $60-80, and each person would spend 800-1000 hours (approx.). The cost per developer turns out to be $48,000-80,000. Hence for 2 developers, it would be $96,000-160,000.*

3. UI/UX Designer - 1 person

For a design intuitive interface requiring minimal training, color-coding schemes, accessibility compliance, and ensuring consistent user experience, we require a UI/UX designer. *We are estimating an hourly rate of $30-40, and each person would spend 800-1000 hours (approx.). The cost per developer turns out to be $24,000-40,000.*

4. QA/Test Engineer - 1 person

We are aiming to achieve more than 80% of unit test coverage requirement, automated integration testing for CRUD and sharing flows, performance testing (2-second load time, 1-second operations). *We are estimating an hourly rate of $45-65, and each person would spend 600-800 hours (approx.). The cost per test engineer turns out to be $27,000-52,000.*

5. DevOps Engineer - 1 person

A DevOps Engineer on our team will be responsible for setting up GitHub CI/CD pipeline, configuring automated testing, deploying of infrastructure, backup systems, and also monitoring. *We are estimating an hourly rate of $60-70, and each person would spend 300-400 hours (approx.). The cost per engineer turns out to be $18,000-28,000.*

We are estimating a Total Development Cost of $305,000 - $540,000. This would be averaging to ~$350,000 for a team of 8 people over 6-9 months of development.

Post Launch Costs

This is an additional inclusion of end-user training. Calendar would require a user manual to be created, if possible, an interactive onboarding guide where the user can see everything step-by-step. The cost of that would be $3,000 (approx.). We will also have troubleshooting guides and FAQs for support. After the launch of our product, we will incur costs for support, bug fixing and also minor updates summing up to around $6400.

Category	**Costs ($)**
Development Team	350,000
Post-launch requirements	9000
Total	359,000

Assumptions made: *We assumed US market rates for the developers and the standard 40 hour work week. This extended our development timeline to around 6-8 months for initial release.*

Test Plan

a) **(Describe the test plan for testing a minimum of one unit of your software.)**

We will focus our unit testing on a critical piece of logic: the event conflict detection. This unit is a function, which we'll call find_event_conflict. This function is responsible for checking if a newly proposed event's time range overlaps with any existing events already in the user's calendar.

Our test plan for this unit involves:

1. To verify that the find_event_conflict function accurately identifies time-blocking conflicts and correctly permits non-conflicting events.
2. We will use automated unit tests written in Python using the pytest framework.
3. This test plan covers one specific function. We will test it in isolation by providing it with a new event and a list of existing events.
4. We will employ a black-box testing strategy by focusing on inputs (event times) and expected outputs (conflict found or not found). We will cover several scenarios:
   - No existing events (empty calendar).
   - No conflict (new event is clearly before or after all existing events).
   - A direct, full overlap (new event is inside an existing event).
   - A partial overlap (new event starts before an existing one but ends during it).
   - An "engulfing" overlap (new event starts before an existing one and ends after it).
   - A boundary case (a new event starts *exactly* when another event ends; this should *not* be a conflict).

b) **(As evidence, write a code for one unit (a method for example) of your software in a programming language of your choice, then use an automated testing tool (such as JUnit for a Java unit) to test your unit and present results.)**

```
from datetime import datetime

def find_event_conflict(new_event, existing_events):
    new_start = new_event['start']
    new_end = new_event['end']

    for old_event in existing_events:
        old_start = old_event['start']
        old_end = old_event['end']
        # check overlap
        if (new_start < old_end) and (new_end > old_start):
            return old_event # conflicting event
    return None
```

```
~ ❯ pytest calendar_event_conflict.py
============================================ test session starts ============================================
platform linux -- Python 3.13.7, pytest-8.4.2, pluggy-1.6.0
benchmark: 4.0.0 (defaults: timer=time.perf_counter disable_gc=False min_rounds=5 min_time=0.000005 max_time=1.
0 calibration_precision=10 warmup=False warmup_iterations=100000)
rootdir: /home/kamui
plugins: anyio-4.11.0, cov-6.1.1, asyncio-0.26.0, benchmark-4.0.0
asyncio: mode=Mode.STRICT, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 6 items

calendar_event_conflict.py ......                                                                      [100%]

============================================= 6 passed in 0.02s =============================================
```

c) **(Clearly define what test case(s) are provided for testing purposes and what results are obtained. (Ch 8))**

The "Existing Events" list for all tests (except Test Case 6) is:

- Event 1: 'Team Meeting' (10:00 - 11:00)
- Event 2: 'Lunch Break' (12:00 - 13:00)

Test Function	**Description**	**Input (New Event Time)**	**Expected Result**
test_no_conflict	Event occurs after all existing events.	14:00 - 15:00	None
test_direct_overlap	Event starts during an existing event.	10:30 - 11:30	Conflict
test_partial_overlap_start	Event starts before and ends during an existing event.	09:30 - 10:30	Conflict ('Team Meeting')
test_engulfing_overlap	Event starts before and ends after an existing event.	09:00 - 11:30	Conflict
test_boundary_no_conflict	Event starts exactly when an existing event ends.	11:00 - 12:00	None

test_empty_calendar	Event is added to an empty calendar.	09:00 - 10:00	None

Comparison of Work with Similar Designs

Features	Google Calendar	Chronos
View Modes	Has different view modes (Day, Week, Month, Year, Schedule/Agenda, 4 days) Also has a different view mode for specifically tasks as a checklist where tasks can be ordered by priority too Can choose to make your calendars public or private for other users to import/view your calendar Can toggle the display of categories	Has less view modes (only Day, Week, Month, Agenda) By default, the calendar will be private and inaccessible to all other users unless you choose to share the calendar to specific users. Unlike in Google Calendar, you won't be able to set your calendar public and let users view your calendar on their own.
Events	Can add events with start and end times, add weekly periodical events, edit & delete events, has event alerts, add/delete event categories, make categories different colors Although you can use Google Calendar's "Find a time" tool in the "more options" mode when creating an event, this only lets you manually find available times instead of notifying you of time conflicts when creating events. However, Google Calendar has even more features when creating events: ● Can add long descriptions and attachments from Google Drive ● Create a Google Meet video conferencing link for the event	Can add events with start and end times, add weekly periodical events, edit & delete events, has event alerts, add/delete event categories, make categories different colors However, Chronos can also check time conflicts whenever events are made with start and end times. So instead of manually finding available times, Chronos will instead ask you to confirm creating an event with a specified time range if it conflicts with another event EX: ("Event Conflict: [Event name and time]. Would you like to continue to add event?")

	● Label yourself as "free" or "busy" during the event time ● Add location for event ● Make it an appointment ● Set the event to a different timezone ● Create and view a guest list	
Share	Send event to other users (requiring their permission) through the internet, but also has the ability to: ● Grant users permission to edit event details ● Add them to a guest list ● Invite them to a Google Meets	Send event to other users (requiring their permission) through the internet so users are only able to view each others calendar
Functionality	Zoom in/out, and scroll support when necessary	Zoom in/out, and scroll support when necessary
Other additional features	Holidays are in special colors (automatically imported calendar into your calendar) Has appointment booking ability where users can create a booking page so other users can easily schedule an appointment with each other ● Also updates to avoid schedule conflicts You can import other calendars ● Other users calendars ● Browse and add other calendars (ex: different religions and their holidays, sports, phases of the moon) ● From a URL or import it yourself manually	Holidays & weekends are in special colors. Google Calendar doesn't separate out their weekends from the weekday, but in Chronos, the weekend will be visually a different color.

Features	**Microsoft Outlook**	**Chronos**
View Modes	**Modes** - Day, Week, Month, Scheduling Assistant, Calendar Overlay, Board View. **Layering -** Multiple calendars on top of each other to see combined schedules at once **Board View -** Add calendars, tasks, and notes to a board view for project management focus S**cheduling Assistant -** Shows availability and conflicts when creating meetings with multiple attendees	**Modes** - Day, Week, Month, Agenda **Agenda View -** Dedicated view for events and tasks in list format. Can toggle the display of categories By default, the calendar will be private and inaccessible to all other users unless you choose to share the calendar to specific users. Unlike Outlook, you won't be able to delegate access or have multiple permission tiers.
Events	Event Actions - 1. Add events with start and end times 2. Add recurring events with advanced patterns (edit series from specific instance onwards), edit & delete events 3. Customizable reminders (with snooze), add/delete event categories 4. Uses conditional formatting for automatic color coding However, Outlook does NOT automatically detect time conflicts - you must use the Scheduling Assistant to manually check availability when inviting others While creating events, users can add descriptions and attachments from OneDrive, and also create Microsoft Teams meeting links automatically. They can set working hours and location for hybrid work scenarios and in turn, mark time as Free/Busy/Out of	Event Actions - 1. Add events with start and end times 2. Add weekly recurring events, edit and delete events 3. Event reminders, add/delete event categories, 4. Automatic color codes by category However, Chronos automatically checks time conflicts whenever events are created with start and end times. Instead of manually finding available times, Chronos will ask you to confirm creating an event if it conflicts with another event. *Ex: ("Event Conflict: [Event name and time]. Would you like to continue to add event?")* Chronos focuses on core event features: • Basic title, description, start/end time, date • Location field (optional) • Recurring events with customizable day patterns and date ranges

	Office/Tentative. Users can add location with map integration and even set events to different time zones. Guest lists with attendee tracking can also be made. • **Meeting Insights (AI-powered) -** Recommends relevant emails, files, and info before meetings • **Keep declined events -** Can keep declined events visible on calendar while showing as free • Supports recurring meeting modifications without breaking series history	• Up to 1,000 events per user supported • Confirmation prompts for edit/delete actions
Sharing	Lets user share calendars with extensive permission options: • **View-only access -** Recipients can see events but not edit • **Edit access -** Recipients can modify events • **Delegate access -** Others can respond to meeting requests on your behalf Send event invitations through email with attendee tracking (accepted/declined/tentative) and the ability to propose alternative meeting times. Users can also add to guest lists, integrate with Microsoft Teams for virtual meetings and are able to get real-time updates across all shared calendars. **SharePoint Calendar Integration -** View and edit team calendars, even when working offline, with automatic sync when reconnected	Send calendar sharing invitations to other users (requiring their permission) through the internet **Basic sharing model -** Users can grant view or edit permissions. Shared users can see events and schedules. Permission-based access control **Simpler than Outlook -** • No delegate access feature • No meeting response tracking • No alternative time proposals • No guest list management • Time zone automatically adjusts for shared events across users in different locations
Functionality	Zoom in/out and scroll support when necessary	Zoom in/out and scroll support when necessary

	Responsive design across Windows, Mac, iOS, Android, and Web **Offline access -** Full offline sync capabilities - can work offline with cached data and sync when reconnected **Real-time sync -** Changes made on one device instantly appear on all devices **Search functionality -** Advanced search across all calendars and events	**Responsive layouts -** Supports screen sizes from 320×568 px and larger **Offline access -** Caches up to 3 months of events locally (max 50 MB) for offline viewing **Performance optimized -** • Calendar views load within 2 seconds • Event operations complete within 1 second **Storage limit -** 10 MB per user (excluding attachments)

<u>Conclusion</u>

a) **Evaluation of your work**
   i) Describe any changes that you needed to make (if any), if things have deviated from what you had originally planned.
   ii) Give justification for such changes

Previously, details about our calendar application were fairly vague and this presented itself as a problem once we had to work on the project scheduling portion. To specify and narrow down the purpose and application of our calendar software, we decided to add the assumption that our software development team of 7 would be developing a calendar application to be integrated into a mid-sized company's employee-only system. Because of this, it made it much less generalized to work on the other parts of the report. For example, because we know the size of the team and what the calendar application would be for, this helped with calculating the function point and effort which then gave a clearer picture of what the project schedule would look like and what the costs of the project (especially for personnel) would be.

During the research portion as well, specifically during the project scheduling point, we decided to base our schedule and plans around an agile process model that combines the incremental process model and the iterative development method. Initially in Deliverable #1, we decided to use the waterfall model as it was under the assumption that we were working in our novice 7-member student team with clearly outlined goals and requirements. This change to not work under a waterfall model was necessary as we were now planning this project under the assumption that it's for a company. With our client being a company, this could mean possibly having unstable requirements that can change depending on user demand and feedback. Because of this, a combination incremental and iterative model would be much more efficient to plan under.

References

[1] Indeed Editorial Team, “Business Sizes: Classifications and Characteristics,” Indeed Career Guide, Aug. 11, 2021. https://www.indeed.com/career-advice/career-development/business-sizes

[2] “Ratio of IT Staff to Employees | Workforce.com>,” Workforce.com. https://workforce.com/news/ratio-of-it-staff-to-employees

[3] “LinkedIn,” *Linkedin.com*, 2025. https://www.linkedin.com/pulse/how-long-does-take-build-software-realistic-timelines-d4vkc/

[4] Vadim Kravcenko, “Rules of Thumb for Software Development Estimations,” *Vadim Kravcenko*, May 2023. https://vadimkravcenko.com/shorts/project-estimates/

[5] “Estimating Software Development Projects: 9 Techniques,” *Atomicobject.com*, 2025. https://atomicobject.com/client-resources/better-custom-software-estimates

[6] “How Much Can a Software Engineer Make in Dallas – 2025 Guide,” *Smu.edu*, 2025. https://techpro.smu.edu/blog/how-much-can-a-software-engineer-make-in-dallas---2025-guide

[7] Apple Inc., "MacBook Pro 16-inch," Apple Store, 2025. https://www.apple.com/shop/buy-mac/macbook-pro (accessed Oct. 24, 2025).

[8] Apple Developer, "Testing your app on multiple devices," Apple Developer Documentation, 2025. https://developer.apple.com/documentation/xcode/testing-your-app-on-multiple-devices (accessed Nov. 2, 2025).

[9] Android Developers, "Test your app," Google Android Developers Guide, 2025. https://developer.android.com/studio/test (accessed Nov. 7, 2025).

[10] S. R. Choudhary, A. Gorla, and A. Orso, "Automated test input generation for Android: Are we there yet?" in Proc. 30th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), 2015, pp. 429-440.

[11] Apple Inc., "iPhone," Apple Store, 2025. https://www.apple.com/iphone/ (accessed Oct. 25, 2025).

[12] Apple Inc., "iPad," Apple Store, 2025. https://www.apple.com/ipad/ (accessed Oct. 26, 2025).

[13] Google LLC, "Google Pixel 9," Google Store, 2025. https://store.google.com/product/pixel_9 (accessed Oct. 31, 2025).

[14] Samsung Electronics, "Galaxy S24 and Galaxy Tab S9," Samsung Official Store, 2025. https://www.samsung.com/ (accessed Oct. 30, 2025).

[15] Dell Technologies, "UltraSharp U3223QE 32 4K USB-C Hub Monitor," Dell Official Store, 2025. https://www.dell.com/en-us/shop/dell-ultrasharp-32-4k-usb-c-hub-monitor-u3223qe/apd/210-bdph (accessed Oct. 30, 2025).

[16] Logitech International, "MX Keys and MX Master 3S," Logitech Official Store, 2025. https://www.logitech.com/ (accessed Nov. 1, 2025).
[17] CalDigit, "Thunderbolt Station 4 (TS4)," CalDigit Official Store, 2025. https://www.caldigit.com/thunderbolt-station-4/ (accessed Nov. 2, 2025).
[18] Dell Technologies, "PowerEdge T550 Tower Server," Dell Business Direct, 2025. https://www.dell.com/en-us/shop/dell-poweredge-servers/sf/poweredge-t-servers (accessed Oct. 31, 2025).
[19] Synology Inc., "DiskStation DS1621+," Synology Products, 2025. https://www.synology.com/en-us/products/DS1621+ (accessed Nov. 1, 2025).
[20] Samsung Electronics, "T7 Portable SSD," Samsung Official Store, 2025. https://www.samsung.com/us/computing/memory-storage/portable-solid-state-drives/ (accessed Nov. 7, 2025).
[21] M. Czerwinski, G. Smith, T. Regan, B. Meyers, G. Robertson, and G. Starkweather, "Toward characterizing the productivity benefits of very large displays," in Proc. INTERACT 2003, 2003, pp. 9-16.
[22] A. N. Meyer, L. E. Barton, G. C. Murphy, T. Zimmermann, and T. Fritz, "The work life of developers: Activities, switches and perceived productivity," IEEE Trans. Softw. Eng., vol. 43, no. 12, pp. 1178-1193, Dec. 2017.
[23] B. W. Boehm, "Software engineering economics," IEEE Trans. Softw. Eng., vol. SE-10, no. 1, pp. 4-21, Jan. 1984.
[24] The cost of developing a custom calendar app in 2024 | by sphinx | medium, https://medium.com/@sphinxshivraj/the-cost-of-developing-a-custom-calendar-app-in-2024-7e8c05b9dc39 (accessed Nov. 22, 2025).
[25] “Front end development cost and key factors in 2025,” Custom Software, AI & Web Development Services Company, https://enqcode.com/blog/front-end-development-cost-and-key-factors (accessed Nov. 21, 2025).
[26] “How to create a calendar app: Key features, cost, Tech Stack,” Cleveroad Inc. - Web and App development company, https://www.cleveroad.com/blog/create-a-calendar-app/ (accessed Nov. 21, 2025).
[27] Front end vs back end - difference between application development - AWS, https://aws.amazon.com/compare/the-difference-between-frontend-and-backend/ (accessed Nov. 22, 2025).
[28] “What is cloud hosting? benefits and risks | google cloud,” Google, https://cloud.google.com/learn/what-is-cloud-hosting (accessed Nov. 21, 2025).
[29] H. Washizaki, “Software engineering body of knowledge (SWEBOK),” IEEE Computer Society, https://www.computer.org/education/bodies-of-knowledge/software-engineering (accessed Nov. 17, 2025).
[30] L. Freifeld, “2023 training industry report,” Training, https://trainingmag.com/2023-training-industry-report/ (accessed Nov. 17, 2025).

[31] “Wages and the Fair Labor Standards Act,” DOL, https://www.dol.gov/agencies/whd/flsa (accessed Nov. 17, 2025).
[32] “Software developer hourly rates | cost to hire software developer | Upwork,” How much does a Software Developer cost?, https://www.upwork.com/hire/software-developers/cost/ (accessed Nov. 17, 2025).
[33] “Software developers, Quality Assurance Analysts, and Testers,” U.S. Bureau of Labor Statistics, https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm (accessed Nov. 17, 2025).
[34] CloudM, “Top 5 features of 365 outlook calendar,” CloudM, https://www.cloudm.io/resources/blog/top-5-features-of-365-outlook-calendar/ (accessed Nov. 17, 2025).
[35] B. Lee, New version of outlook: Updated design, integration with loop and Todo, and intelligent assistance – 4sysops, https://4sysops.com/archives/new-version-of-outlook-updated-design-integration-with-loop-and-todo-and-intelligent-assistance/ (accessed Nov. 17, 2025).
[36] “Microsoft,” Microsoft Support, https://support.microsoft.com/en-us/office/introduction-to-the-outlook-calendar-d94c5203-77c7-48ec-90a5-2e2bc10bd6f8 (accessed Nov. 17, 2025).